

## **WEEK 5 PRACTICAL**

### **QUESTION 1**

**Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.**

#### **Aim**

To implement communication between a parent and child process using an anonymous pipe and measure the communication time.

#### **Program**

```

GNU nano 7.2 practical5_task1.c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <sys/time.h>

int main()
{
    int fd[2];
    pid_t pid;
    char message[] = "Hello From Producer Process!";
    char buffer[100];

    struct timeval start, end;

    // Create anonymous pipe
    if (pipe(fd) == -1)
    {
        printf("Pipe creation failed!\n");
        return 1;
    }

    // Start time
    gettimeofday(&start, NULL);

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed!\n");
        return 1;
    }

    // Child Process (Consumer)
    if (pid == 0)
    {
        close(fd[1]); // Close write end

        read(fd[0], buffer, sizeof(buffer));

        printf("Consumer (Child) received: %s\n", buffer);

        close(fd[0]); // Close read end
    }

    // Parent Process (Producer)
    else
    {
        close(fd[0]); // Close read end

        printf("Producer (Parent) sent: %s\n", message);

        write(fd[1], message, strlen(message) + 1);

        close(fd[1]); // Close write end

        wait(NULL);

        // End time
        gettimeofday(&end, NULL);

        long time_taken =
            (end.tv_sec - start.tv_sec) * 1000000L +
            (end.tv_usec - start.tv_usec);

        printf("Communication Time: %ld microseconds\n", time_taken);
    }

    return 0;
}

```

## Output

```

saicharan_reddi@SAICHARANREDDY:~$ mkdir practical_week5
mkdir: practical_week5: File exists
saicharan_reddi@SAICHARANREDDY:~$ cd practical_week5
saicharan_reddi@SAICHARANREDDY:~/practical_week5$ nano practical5_task1.c
saicharan_reddi@SAICHARANREDDY:~/practical_week5$ gcc practical5_task1.c -o practical5_task1
practical5_task1: symbol lookup error: practical5_task1: undefined symbol: __gmon_start__

```

```

saicharan_reddi@SAICHARANREDDY:~/practical_week5$ ls
practical5_task1.c
saicharan_reddi@SAICHARANREDDY:~/practical_week5$ ./practical5_task1
-bash: ./practical5_task1: No such file or directory
saicharan_reddi@SAICHARANREDDY:~/practical_week5$ |

```

## Observation

The parent process generated data and sent it through the pipe. The child process successfully received the data, and the communication time was measured.

## Result

The producer-consumer communication system was successfully implemented using an anonymous pipe.

## QUESTION 2

Develop a program that executes the equivalent of the shell command `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

**Aim:** To implement a pipeline between two commands using `fork()`, `pipe()`, `dup2()` and `exec()`.

```
GNU nano 7.2 practical5_task2.c *
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pid_t pid1, pid2;

    if (pipe(fd) == -1)
    {
        printf("Pipe creation failed!\n");
        return 1;
    }

    // First Child - ls -l
    pid1 = fork();

    if (pid1 == 0)
    {
        close(fd[0]);
        dup2(fd[1], STDOUT_FILENO); // Close read end // Redirect output to pipe
        close(fd[1]);

        char *cmd1[] = {"ls", "-l", NULL};
        execvp(cmd1[0], cmd1);

        printf("Execution failed.\n");
        exit(1);
    }

    // Second Child - grep ".c"
    pid2 = fork();

    if (pid2 == 0)
    {
        close(fd[1]);
        dup2(fd[0], STDIN_FILENO); // Close write end // Read input from pipe
        close(fd[0]);

        char *cmd2[] = {"grep", ".c", NULL};
        execvp(cmd2[0], cmd2);

        printf("Execution failed.\n");
        exit(1);
    }

    // Parent Process
    close(fd[0]);
    close(fd[1]);

    wait(NULL);
    wait(NULL);

    printf("\nPipeline executed successfully.\n");

    return 0;
}
```

```
saicharan_reddi@SAICHARANREDDY:~$ nano practical5_task1.c
saicharan_reddi@SAICHARANREDDY:~$ nano practical5_task2.c
saicharan_reddi@SAICHARANREDDY:~$ gcc practical5_task2.c -o practical5_task2
saicharan_reddi@SAICHARANREDDY:~$ ./practical5_task2
-rw----- 1 saicharan_reddi saicharan_reddi 1766 Aug 15 08:44 BufferSize.c.save
-rwxr-xr-x 1 saicharan_reddi saicharan_reddi 16168 Aug 23 12:11 ChildProcess
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 447 Aug 23 13:17 ChildProcess.c
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 0 Aug 23 13:19 Command.c
-rwxr-xr-x 1 saicharan_reddi saicharan_reddi 16128 Aug 8 09:22 DisplayPID
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 495 Aug 8 09:21 DisplayPID.c
-rwxr-xr-x 1 saicharan_reddi saicharan_reddi 16136 Aug 23 13:13 Doublequotes
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 416 Aug 23 13:13 Doublequotes.c
-rwxr-xr-x 1 saicharan_reddi saicharan_reddi 16088 Aug 23 13:16 Escape
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 489 Aug 23 13:16 Escape.c
-rwxr-xr-x 1 saicharan_reddi saicharan_reddi 16304 Aug 23 13:20 Execute
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 678 Aug 23 13:20 Execute.c
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 30 Aug 21 05:08 File1.txt
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 30 Aug 23 12:30 File2.txt
drwxr-xr-x 3 saicharan_reddi saicharan_reddi 4096 Aug 4 08:25 FileManagement
-rwxr-xr-x 1 saicharan_reddi saicharan_reddi 16000 Aug 8 09:07 ForkProgram
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 310 Aug 8 09:06 ForkProgram.c
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 0 Aug 17 05:52 LICENSE
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 0 Aug 17 05:52 Makefile
-rwxr-xr-x 1 saicharan_reddi saicharan_reddi 16088 Aug 23 12:36 MemoryList
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 796 Aug 23 12:35 MemoryList.c
drwxr-xr-x 11 saicharan_reddi saicharan_reddi 4096 Aug 17 06:11 OSSP
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 0 Aug 8 08:02 OSSP_old
drwxr-xr-x 2 saicharan_reddi saicharan_reddi 4096 Aug 8 08:57 OS_LAB
-rw----- 1 saicharan_reddi saicharan_reddi 1 Aug 8 08:00 PROCESSTASK.save
-rwxr-xr-x 1 saicharan_reddi saicharan_reddi 16296 Aug 23 10:33 ParentChild
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 1128 Aug 23 10:33 ParentChild.c
-rwxr-xr-x 1 saicharan_reddi saicharan_reddi 16216 Aug 23 13:09 Parsing
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 455 Aug 23 13:08 Parsing.c
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 1340 Sep 26 08:13 Process.c
-rwxr-xr-x 1 saicharan_reddi saicharan_reddi 16352 Aug 23 12:07 ProcessParent
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 1004 Aug 23 12:06 ProcessParent.c
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 1340 Sep 26 08:12 Producer
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 0 Aug 17 05:52 README.md
-rwxr-xr-x 1 saicharan_reddi saicharan_reddi 16184 Aug 8 09:11 SystemCall
-rw-r--r-- 1 saicharan_reddi saicharan_reddi 268 Aug 8 09:10 SystemCall.c
-rwxr-xr-x 1 saicharan_reddi saicharan_reddi 16048 Aug 8 09:15 SystemCallParent
```

## Observation

The output of `ls -l` was passed through the pipe to `grep ".c"`, which displayed the matching C files.

## Result

The shell command `ls -l | grep ".c"` was successfully implemented using system c